

# Sprawozdanie 2 - Zaawansowane techniki optymalizacji funkcji ciągłych

Jakub Jaśków 268416, Justyna Ziemichód 268431

1 sierpnia 2026

## Spis treści

|          |                                                                     |          |
|----------|---------------------------------------------------------------------|----------|
| <b>1</b> | <b>Cel ćwiczenia</b>                                                | <b>2</b> |
| <b>2</b> | <b>Środowisko i zastosowane algorytmy</b>                           | <b>2</b> |
| <b>3</b> | <b>Analiza badanej przestrzeni poszukiwań</b>                       | <b>2</b> |
| <b>4</b> | <b>Zadanie A: Własne operatory dla algorytmu genetycznego</b>       | <b>3</b> |
| 4.1      | Szczegółowy opis zaproponowanych operatorów . . . . .               | 3        |
| 4.1.1    | Własna Selekcja: Turniejowa z progiem (Tournament Selection) . .    | 3        |
| 4.1.2    | Własne Krzyżowanie: Uśredniające (Arithmetic Crossover) . . . .     | 3        |
| 4.1.3    | Własna Mutacja: Gaussowska (Gaussian Mutation) . . . . .            | 4        |
| 4.2      | Porównanie wyników . . . . .                                        | 4        |
| <b>5</b> | <b>Zadanie D: Badanie algorytmu PSO</b>                             | <b>5</b> |
| 5.1      | Rozmiar roju ( $s$ ) . . . . .                                      | 5        |
| 5.2      | Współczynnik bezwładności ( $w$ ) . . . . .                         | 5        |
| 5.3      | Liczba iteracji ( $maxit$ ) . . . . .                               | 6        |
| 5.4      | Współczynniki: kognitywny ( $c_p$ ) i społeczny ( $c_g$ ) . . . . . | 6        |
| <b>6</b> | <b>Podsumowanie</b>                                                 | <b>7</b> |
| <b>A</b> | <b>Załącznik: Kod źródłowy w języku R</b>                           | <b>8</b> |

# 1 Cel ćwiczenia

Celem laboratorium było rozszerzenie analizy algorytmów optymalizacji globalnej dla wielomodalnych funkcji ciągłych. W ramach niego zrealizowano dwa zadania:

1. **Zadanie A:** Zaproponowanie i implementacja własnych operatorów genetycznych (selekcji, krzyżowania i mutacji) dla klasycznego Algorytmu Genetycznego (GA) oraz porównanie ich skuteczności z wariantami domyślnymi.
2. **Zadanie D:** Zastosowanie algorytmu optymalizacji rojem cząstek (PSO – *Particle Swarm Optimization*) do minimalizacji wybranej funkcji celu oraz empiryczne zbadanie wpływu jego wewnętrznych parametrów kontrolnych na zbieżność.

# 2 Środowisko i zastosowane algorytmy

## Narzędzia i biblioteki

Całość badań oraz wizualizację danych przeprowadzono w środowisku programistycznym **R**. Do implementacji i testowania algorytmu genetycznego wykorzystano pakiet **GA**, natomiast do optymalizacji rojem cząstek zastosowano pakiet **pso**, który zapewnia zaawansowane mechanizmy heurystyczne dla przestrzeni ciągłych.

## Krótką charakterystyka algorytmów

**Algorytm Genetyczny (GA)** to metaheurystyka inspirowana procesem ewolucji biologicznej. Algorytm utrzymuje populację potencjalnych rozwiązań (osobników), które w każdej iteracji (pokoleniu) podlegają ocenie na podstawie funkcji przystosowania. Najlepsze osobniki są wybierane (selekcja) i łączone w pary (krzyżowanie) w celu wygenerowania potomstwa. Dodatkowo, z pewnym prawdopodobieństwem zachodzi mutacja, która wprowadza losowe zmiany, chroniąc populację przed utknięciem w minimum lokalnym.

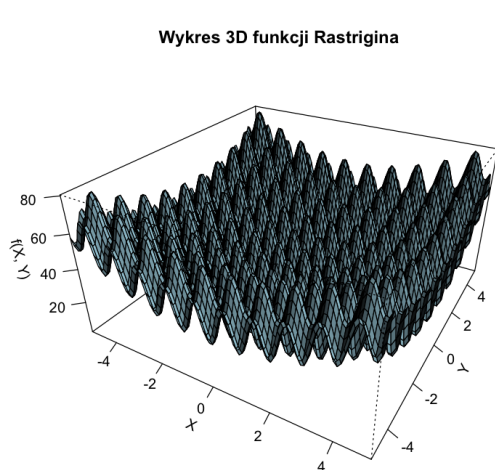
**Optymalizacja Rojem Cząstek (PSO)** to algorytm oparty na inteligencji stadnej (np. zachowaniu ławic ryb). W przestrzeni poszukiwań umieszczana jest populacja cząstek. Każda cząstka pamięta swoje najlepsze dotychczasowe położenie ( $p_{best}$ ) oraz zna najlepsze położenie znalezione przez cały rój ( $g_{best}$ ). W każdej iteracji cząstka aktualizuje swój wektor prędkości i przemieszcza się, dążąc w stronę kompromisu między własnym doświadczeniem a wiedzą stada, z uwzględnieniem parametru bezwładności ( $w$ ).

# 3 Analiza badanej przestrzeni poszukiwań

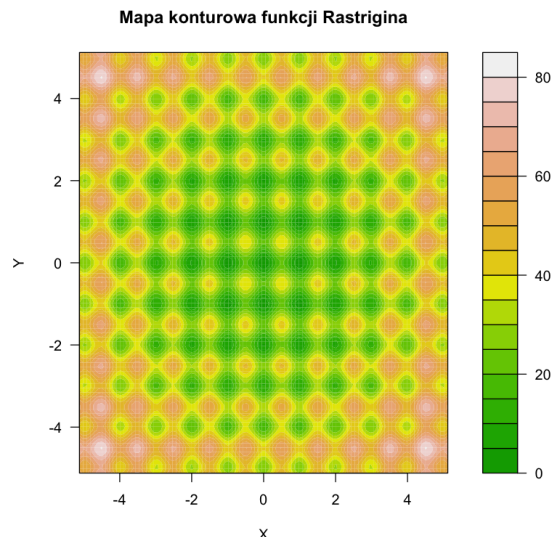
Przed przystąpieniem do weryfikacji skuteczności algorytmów, przeanalizowano topologię wybranej do testów funkcji Rastrigina. Równanie funkcji dla przestrzeni dwuwymiarowej ma postać:

$$f(x, y) = 20 + x^2 - 10 \cos(2\pi x) + y^2 - 10 \cos(2\pi y)$$

Dziedzina poszukiwań została ograniczona do przedziału  $x, y \in [-5, 12, 5, 12]$ . Minimum globalne tej funkcji znajduje się w punkcie  $(0, 0)$  i wynosi  $f(0, 0) = 0$ .



Rysunek 1: Wykres powierzchniowy 3D.



Rysunek 2: Mapa konturowa (widok z góry).

## 4 Zadanie A: Własne operatory dla algorytmu genetycznego

Do testów wybrano funkcję Rastrigina, ponieważ jej silnie wielomodalny charakter (rozległa siatka minimumów lokalnych) doskonale weryfikuje zdolność nowo zaprojektowanych operatorów do zachowania balansu między globalną eksploracją a lokalną eksploatacją.

### 4.1 Szczegółowy opis zaproponowanych operatorów

Domyślne operatory w bibliotece **GA** opierają się na selekcji ruletkowej, jednopunktowym krzyżowaniu i mutacji jednorodnej. Zaimplementowano własne funkcje z wykorzystaniem specyfiki przestrzeni ciągłej:

#### 4.1.1 Własna Selekcja: Turniejowa z progiem (Tournament Selection)

Zaimplementowano mechanizm **selekcji turniejowej**. Z populacji losowana jest ze zwracaniem grupa  $k = 3$  osobników. Do puli rozrodczej przechodzi wyłącznie ten osobnik, którego wartość funkcji przystosowania jest najwyższa. Jest to metoda odporna na problem skalowania i pozwala płynnie sterować presją selekcyjną.

#### 4.1.2 Własne Krzyżowanie: Uśredniające (Arithmetic Crossover)

Zamiast krzyżowania punktowego, zaproponowano krzyżowanie oparte na kombinacji liniowej. Potomek powstaje na odcinku łączącym rodziców:

$$x_{potomek} = \alpha \cdot x_{rodzic1} + (1 - \alpha) \cdot x_{rodzic2}$$

gdzie  $\alpha \in [0, 1]$  jest losowym współczynnikiem dobieranym dla każdej pary. Pozwala to na precyzyjne odnajdywanie minimum "pomiędzy" już odnalezionymi rozwiązaniami.

### 4.1.3 Własna Mutacja: Gaussowska (Gaussian Mutation)

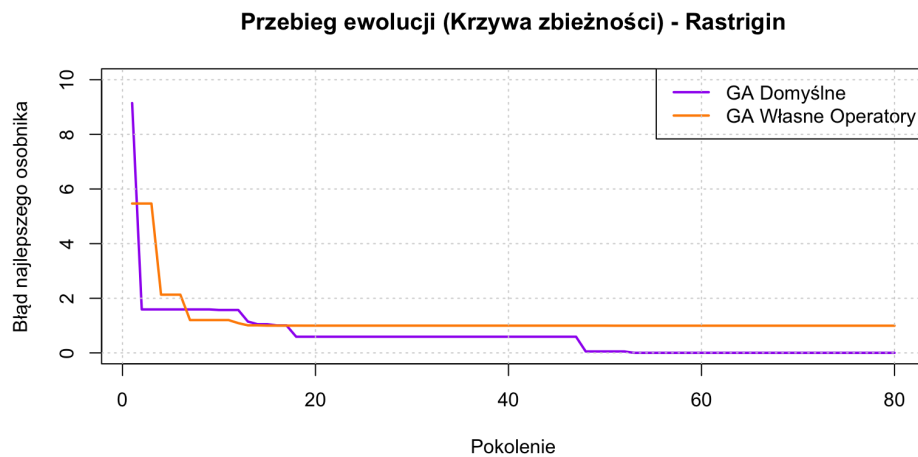
Wylosowany do mutacji gen  $x_i$  jest modyfikowany o losowe zaburzenie:

$$x_i^{nowy} = x_i^{stary} + \mathcal{N}(0, \sigma)$$

gdzie  $\mathcal{N}(0, \sigma)$  to wartość z rozkładu normalnego (w badaniach  $\sigma = 0.5$ ). Pozwala to na delikatne dostrajanie rozwiązania z jednoczesną szansą na wyjście z ekstremum lokalnego w przypadku rzadszych, większych zaburzeń.

## 4.2 Porównanie wyników

Przeprowadzono analizę zbieżności algorytmu z wariantem domyślnym oraz z wariantem wykorzystującym zaimplementowane własne operatory (rozmiar populacji = 50, liczba pokoleń = 80).



Rysunek 3: Przebieg ewolucji (Krzywa zbieżności) - Rastrigin.

Krzywa zbieżności jednoznacznie pokazuje, że domyślne operatory pakietu GA poradziły sobie znacznie lepiej. Fioletowa krzywa zbiega niemal idealnie do zera (odnalezienie globalnego minimum). Z kolei wariant z własnymi operatorami (linia pomarańczowa) uległ przedwczesnej zbieżności i utknął na poziomie błędu rzędu 1, 0.

| Wariant Algorytmu       | Końcowy błąd (po 80 iteracjach) |
|-------------------------|---------------------------------|
| GA - Operatory Domyślne | 0,0015                          |
| GA - Własne Operatory   | 0,9951                          |

Tabela 1: Porównanie błędu końcowego klasycznego GA i GA z własnymi operatorami.

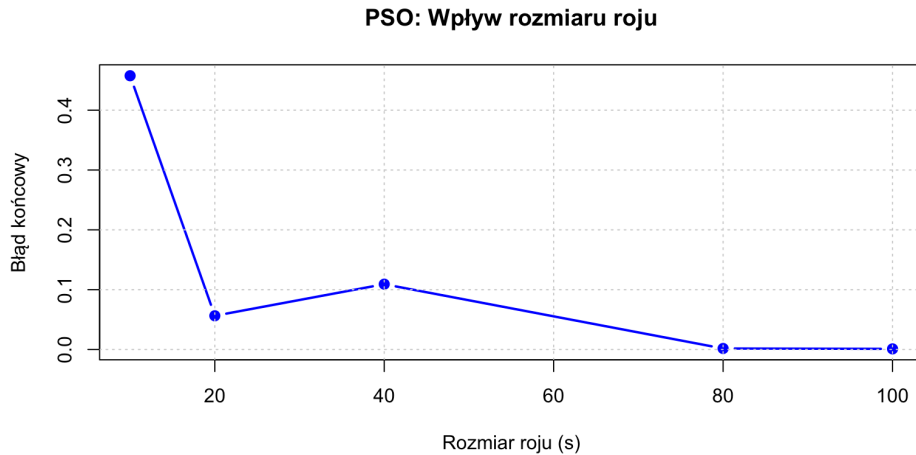
**Wniosek:** Wprowadzenie prostej mutacji Gaussowskiej i krzyżowania arytmetycznego zaburzyło balans eksploracji. Algorytm zaczął zbyt mocno eksploatować lokalne sąsiedztwo, co na silnie wielomodalnej funkcji Rastrigina poskutkowało utknięciem populacji w głębokim minimum lokalnym. Standardowe operatory biblioteki GA posiadają znacznie lepiej zoptymalizowane mechanizmy ucieczki z takich pułapek.

## 5 Zadanie D: Badanie algorytmu PSO

Przeanalizowano wpływ parametrów kontrolnych algorytmu roju cząstek (PSO) minimalizującego funkcję Rastrigina. Wszystkie wyniki przedstawiono jako średni błąd po 10 niezależnych uruchomieniach dla każdego zestawu parametrów.

### 5.1 Rozmiar roju ( $s$ )

Rozmiar roju warunkuje pokrycie przestrzeni w początkowej fazie działania algorytmu.

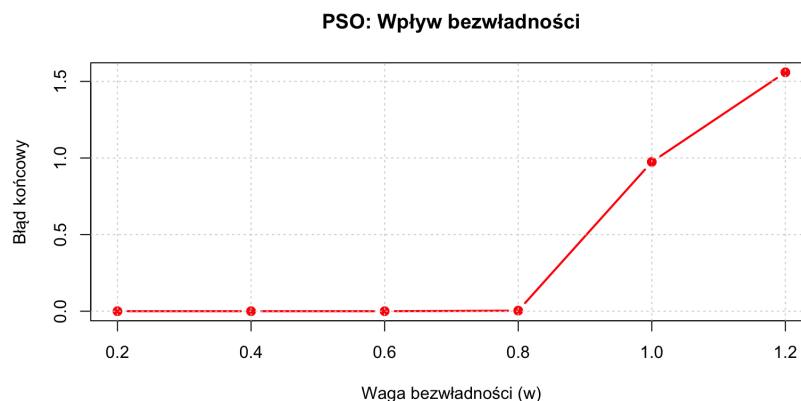


Rysunek 4: Wpływ rozmiaru roju na końcowy błąd.

Wraz ze wzrostem liczby cząstek, błąd maleje niemal do zera. Ze względu na wielomodalny charakter funkcji, rój poniżej 40 osobników zbyt często wpada w minima lokalne.

### 5.2 Współczynnik bezwładności ( $w$ )

Parametr ten opisuje skłonność cząstki do zachowania dotychczasowego pędu.



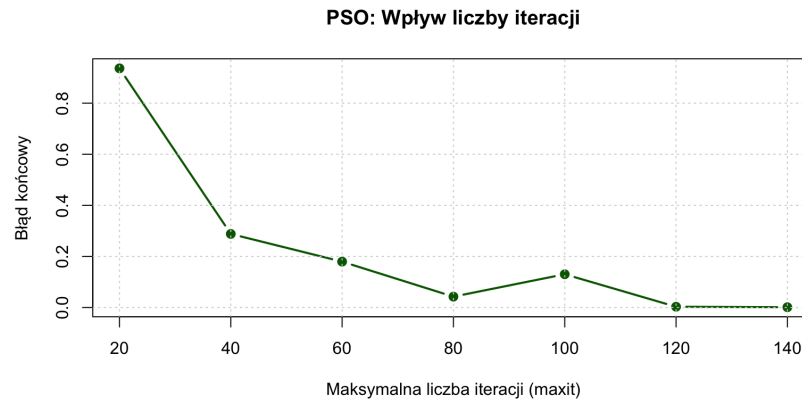
Rysunek 5: Wpływ wagi bezwładności na zbieżność.

Niska bezwładność ( $w \leq 0.8$ ) pozwala rojowi szybko osiadać w ekstremach, gwarantując niskie błędy. Przekroczenie progu  $w > 0.8$  skutkuje rozbieganiem się cząstek – nie

są one w stanie wyhamować nad minimum globalnym, co widać jako drastyczny wzrost błędu końcowego.

### 5.3 Liczba iteracji (*maxit*)

Zbadano proces uczenia się roju w czasie.

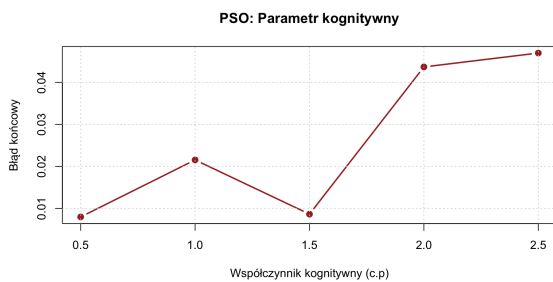


Rysunek 6: Krzywa zbieżności PSO w zależności od dostępnych iteracji.

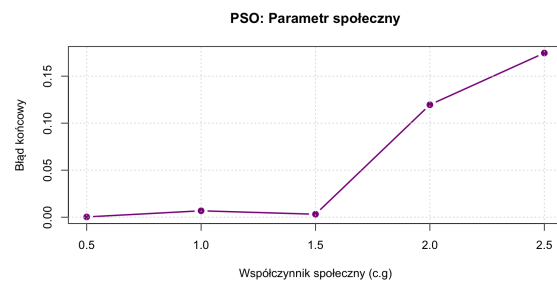
Widoczny jest asymptotyczny spadek błędu. Od około 120 iteracji rój w pełni zbiega do globalnego optimum dla uśrednionych parametrów bazowych algorytmu.

### 5.4 Współczynniki: kognitywny ( $c_p$ ) i społeczny ( $c_g$ )

Współczynnik  $c_p$  decyduje o przyciąganiu cząstki do własnej historii, a  $c_g$  o zaufaniu do wyników reszty roju.



Rysunek 7: Wpływ współczynnika kognitywnego.



Rysunek 8: Wpływ współczynnika społecznego.

Zarówno zbyt wysokie zaufanie wyłącznie do własnej pamięci ( $c_p > 1.5$ ), jak i bezgraniczne "śledzenie tłumu" ( $c_g > 1.5$ ) prowadzą do pogorszenia wyników, wybijając algorytm z balansu eksploracyjno-eksploatacyjnego.

Tabela 2: Szczegółowe dane liczbowe dla badanych parametrów z wygenerowanych wektorów.

| $s$ (Rozmiar roju) | Błąd   | $w$ (Bezwładność) | Błąd   |
|--------------------|--------|-------------------|--------|
| 10                 | 0,4575 | 0,2               | 0,0000 |
| 20                 | 0,0564 | 0,4               | 0,0000 |
| 40                 | 0,1092 | 0,6               | 0,0000 |
| 80                 | 0,0017 | 0,8               | 0,0037 |
| 100                | 0,001  | 1,0               | 0,9744 |
| -                  | -      | 1,2               | 1,5592 |

## 6 Podsumowanie

Przeprowadzone badania pozwoliły na dogłębną analizę działania algorytmów optymalizacji globalnej na przykładzie silnie wielomodalnej funkcji Rastrigina. Z eksperymentów płyną dwa główne wnioski dotyczące badanych metod.

**W przypadku Algorytmu Genetycznego (GA)** samodzielna implementacja operatorów (krzyżowanie arytmetyczne, mutacja Gaussowska) wykazała, jak delikatny jest balans między eksploracją (przeszukiwaniem nowych obszarów) a eksploatacją (zawężaniem poszukiwań). Zastosowane własne metody doprowadziły do przedwczesnej zbieżności i utknięcia populacji w minimum lokalnym. Stanowi to dowód na to, że domyślne operatory wbudowane w pakiety optymalizacyjne są wysoce dopracowane i znacznie skuteczniej radzą sobie z ucieczką z pułapek lokalnych ekstremów.

**W przypadku Optymalizacji Rojem Cząstek (PSO)** algorytm ten okazał się niezwykle skuteczny, ale jednocześnie wrażliwy na dobór parametrów kontrolnych. Kształt funkcji Rastrigina wymusza zastosowanie relatywnie dużej populacji początkowej ( $s \geq 40$ ), aby od samego początku dobrze pokryć przestrzeń poszukiwań. Kluczowym parametrem okazała się waga bezwładności ( $w$ ) – jej optymalna wartość w okolicach 0.8 pozwala na płynne zbieganie roju. Nawet niewielkie odchylenia od tej wartości skutkują zjawiskiem rozbiegania się cząstek lub natychmiastowym utknięciem w pierwszym napotkanym minimum lokalnym.

Ostatecznie można stwierdzić, że optymalizacja funkcji ciągłych o tak skomplikowanej topologii wymaga nie tylko doboru odpowiedniego algorytmu, ale także świadomego procesu strojenia jego parametrów.

## Literatura

- [1] Scrucca L. (2013). *GA: A Package for Genetic Algorithms in R*. Journal of Statistical Software, 53(4), 1-37.
- [2] Bendtsen C. (2012). *pso: Particle Swarm Optimization*. R package version 1.0.3.
- [3] Kennedy, J., & Eberhart, R. (1995). *Particle swarm optimization*. Proceedings of ICNN'95 - International Conference on Neural Networks, vol. 4, 1942-1948.

## A Załącznik: Kod źródłowy w języku R

Poniżej znajduje się kompletny skrypt wykorzystany do generowania wyników i wykresów.

```
1 library(GA)
2 library(pso)
3 set.seed(42)
4
5 # --- 1. FUNKCJE BAZOWE ---
6 rastrigin <- function(x1, x2) { 20 + (x1^2 - 10 * cos(2 * pi * x1)) + (
  x2^2 - 10 * cos(2 * pi * x2)) }
7 fit_ga <- function(x) { -rastrigin(x[1], x[2]) }
8 fit_pso <- function(x) { rastrigin(x[1], x[2]) }
9
10 # --- 2. WLASNE OPERATORY GA ---
11 custom_tourSel <- function(obj, k=3) {
12   pop <- obj@population; fit <- obj@fitness; sel <- rep(NA, obj@popSize)
13   for(i in 1:obj@popSize) { sel[i] <- sample(1:obj@popSize, k)[which.max
    (fit[sample(1:obj@popSize, k)])] }
14   list(population = pop[sel,,drop=FALSE], fitness = fit[sel])
15 }
16 custom_arithCross <- function(obj, par) {
17   p1 <- obj@population[par[1],]; p2 <- obj@population[par[2],]; a <-
    runif(1)
18   list(children = rbind(a*p1 + (1-a)*p2, (1-a)*p1 + a*p2), fitness = c(
    NA, NA))
19 }
20 custom_gaussMut <- function(obj, par) {
21   m <- as.vector(obj@population[par,]); j <- sample(1:length(m), 1)
22   m[j] <- min(max(m[j] + rnorm(1, 0, 0.5), obj@lower[j]), obj@upper[j]);
    m
23 }
24
25 # --- 3. KRZYWA ZBIEZNOSCI GA (Domyslne vs Wlasne) ---
26 cat("Generowanie krzywej zbieznosci GA...\n")
27 ga_default <- ga(type="real-valued", fitness=fit_ga, lower=c
  (-5.12, -5.12), upper=c(5.12, 5.12), popSize=50, maxiter=80, monitor=
  FALSE)
28 ga_custom <- ga(type="real-valued", fitness=fit_ga, lower=c(-5.12, -5.12)
  , upper=c(5.12, 5.12), popSize=50, maxiter=80,
29   selection=custom_tourSel, crossover=custom_arithCross,
  mutation=custom_gaussMut, monitor=FALSE)
30
31 plot(1:80, -ga_default@summary[, "max"], type="l", lwd=2, col="purple",
  ylim=c(0, 10),
32   main="Przebieg ewolucji (Krzywa zbieznosci) - Rastrigin", xlab="
  Pokolenie", ylab="Bład najlepszego osobnika")
33 lines(1:80, -ga_custom@summary[, "max"], lwd=2, col="darkorange")
34 legend("topright", legend=c("GA Domyslne", "GA Wlasne Operatory"), col=c
  ("purple", "darkorange"), lwd=2)
35 grid()
36
37 # --- 4. FUNKCJA TESTUJACA PSO ---
38 run_pso <- function(s=40, maxit=100, w=0.8, cp=1.5, cg=1.5) {
39   mean(sapply(1:10, function(i) {
40     psoptim(runif(2, -5.12, 5.12), fit_pso, lower=c(-5.12, -5.12), upper=
      c(5.12, 5.12),
41       control=list(s=s, maxit=maxit, w=w, c.p=cp, c.g=cg)))$value
42   })))
```



```

43 }
44
45 cat("Obliczanie parametrow PSO...\n")
46
47 # Wykres 1: Rozmiar roju
48 v_s <- c(10, 20, 40, 80, 100); r_s <- sapply(v_s, function(x) run_pso(s=
  x))
49 plot(v_s, r_s, type="b", pch=19, col="blue", lwd=2, main="PSO: Wplyw
  rozmiaru roju", xlab="Rozmiar roju (s)", ylab="Blad koncowy"); grid()
50
51 # Wykres 2: Bezwladnosc
52 v_w <- seq(0.2, 1.2, by=0.2); r_w <- sapply(v_w, function(x) run_pso(w=x
  ))
53 plot(v_w, r_w, type="b", pch=19, col="red", lwd=2, main="PSO: Wplyw
  bezwladnosci", xlab="Waga bezwladnosci (w)", ylab="Blad koncowy");
  grid()
54
55 # Wykres 3: Liczba iteracji
56 v_i <- c(20, 40, 60, 80, 100, 120, 140); r_i <- sapply(v_i, function(x)
  run_pso(maxit=x))
57 plot(v_i, r_i, type="b", pch=19, col="darkgreen", lwd=2, main="PSO:
  Wplyw liczby iteracji", xlab="Maksymalna liczba iteracji (maxit)",
  ylab="Blad koncowy"); grid()
58
59 # Wykres 4: Parametr kognitywny (c.p)
60 v_cp <- seq(0.5, 2.5, by=0.5); r_cp <- sapply(v_cp, function(x) run_pso(
  cp=x))
61 plot(v_cp, r_cp, type="b", pch=19, col="brown", lwd=2, main="PSO:
  Parametr kognitywny", xlab="Wspolczynnik kognitywny (c.p)", ylab="
  Blad koncowy"); grid()
62
63 # Wykres 5: Parametr spoleczny (c.g)
64 v_cg <- seq(0.5, 2.5, by=0.5); r_cg <- sapply(v_cg, function(x) run_pso(
  cg=x))
65 plot(v_cg, r_cg, type="b", pch=19, col="darkmagenta", lwd=2, main="PSO:
  Parametr spoleczny", xlab="Wspolczynnik spoleczny (c.g)", ylab="Blad
  koncowy"); grid()

```